

EECS 31L: INTRODUCTION TO DIGITAL DESIGN LAB LECTURE 4

Salma Elmalaki
salma.elmalaki@uci.edu

The Henry Samueli School of Engineering
Electrical Engineering and Computer Science
University of California, Irvine



LECTURE 4: LOGISTICS

- Lab 2 is assigned - Please work with your TAs during your lab session to debug your code.
- There will be a 15-minute Quiz assigned on Canvas covering contents up till this lecture. Due Friday at 1:59pm (No Late submission).
- One question in this quiz will require you to sign into **Snorkl AI** class with **your UCI email** (check the canvas announcement) and record your explanation for the answer.
- Midterm is on Feb 18 at 5pm on **Respondus Lockdown browser with webcam** (Mock Exam in on canvas to test your setup)

Last Time

- Verilog Operators
- Typical mistakes in Verilog
- Testbench

LECTURE 4: OVERVIEW

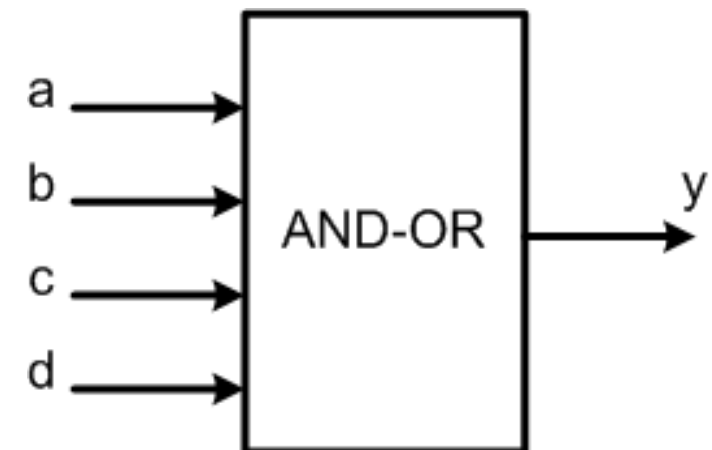
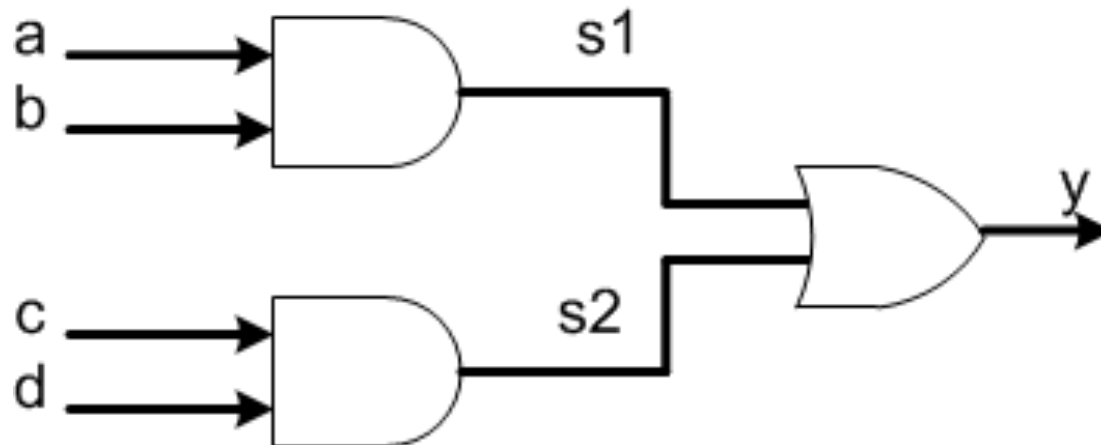
- Data flow description
- `Generate for`: concurrent execution
- Behavioral Description
- Sequential statements
 - if, case, for, while, ...
 - Storage elements
 - DFF
 - JK FF
 - Binary counters
- Blocking vs. non-blocking assignment

DATA FLOW DESCRIPTION

- How data flows from input to output
- Circuit is designed with **concurrent** statements
 - The order of the statements is **not** important
 - The execution of the statements is event-based
 - Useful for combinational circuits modeling
- More realistic circuit modeling if either of the followings are available:
 - The Boolean equation
 - Truth table

DATA FLOW DESCRIPTION - EXAMPLE 1

- AND-OR block
- The circuit of the design



- A Boolean function of the output

$$\rightarrow y = (ab) + (cd)$$

Input				Output
a	b	c	d	Y
1	1	-	-	1
-	-	1	1	1
1	1	1	1	1

VERILOG CODE FOR AND-OR BLOCK

```
module and_or(a,b,c,d,y);
```

```
    input a,b,c,d;
```

```
    output y;
```

```
    wire s1,s2;
```

```
    assign s1 = a & b;
```

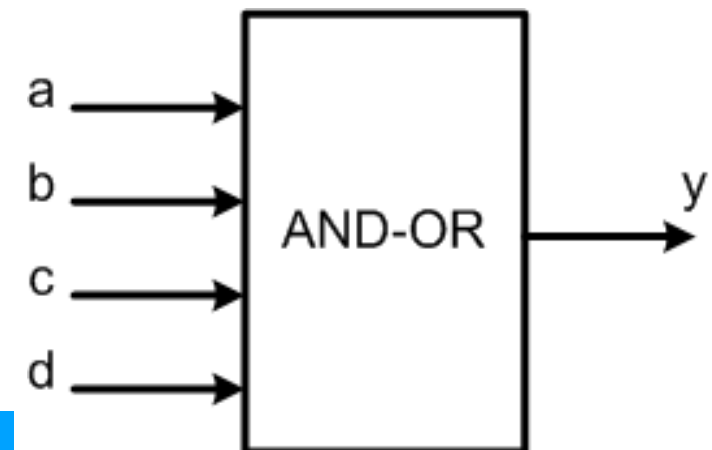
```
    assign s2 = c & d;
```

```
    assign y = s1 | s2;
```

```
endmodule
```

Wire statement here is not necessarily needed since ports by default are wires

Runs concurrently



Input				Output
a	b	c	d	Y
1	1	-	-	1
-	-	1	1	1
1	1	1	1	1

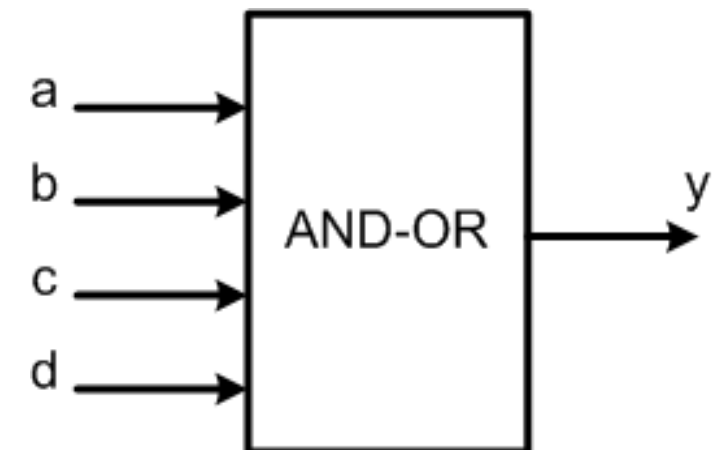
VERILOG CODE FOR AND-OR BLOCK

```
module and_or(a,b,c,d,y);
    input a,b,c,d;
    output y;
```

```
    wire s1,s2;
    assign s1 = a & b;
    assign s2 = c & d;
    assign y  = s1 | s2;
```

```
endmodule
```

Runs concurrently



Input				Output
a	b	c	d	Y
1	1	-	-	1
-	-	1	1	1
1	1	1	1	1

VERILOG CODE FOR AND-OR BLOCK

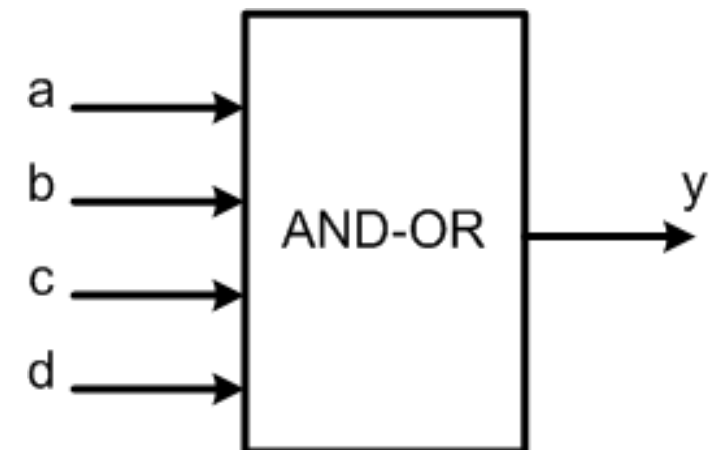
```

module and_or(a,b,c,d,y);
    input a,b,c,d;
    output y;

    wire s1,s2;
    assign y = s1 | s2;
    assign s2 = c & d;
    assign s1 = a & b;

endmodule

```



Input				Output
a	b	c	d	Y
1	1	-	-	1
-	-	1	1	1
1	1	1	1	1

Runs concurrently
Changing order
doesn't matter

CONCURRENT SIGNAL ASSIGNMENT (SUMMARY)

- Signal assignment execution
 - Different from language such as C or Python
 - An event should occur on the right side
 - Event is a change of value of right side
 - For example, from 0 to 1 or even 0 to Z
 - The statement that receives an event is executed first
 - More than one statement can have event at the same time
 - executed concurrently
- Signal statement phases
 - Calculation
 - At the time of event happens
 - Assignment
 - After calculation is done
- **assign** is the keyword

SIMULATION OF AND-OR

• Event at T_0

- On a and b; statement 1 is executed.
- After executing first statement the output s1 is updated
 - No delay; there is an event on the third statement at T_0
 - So y also changes at T_0

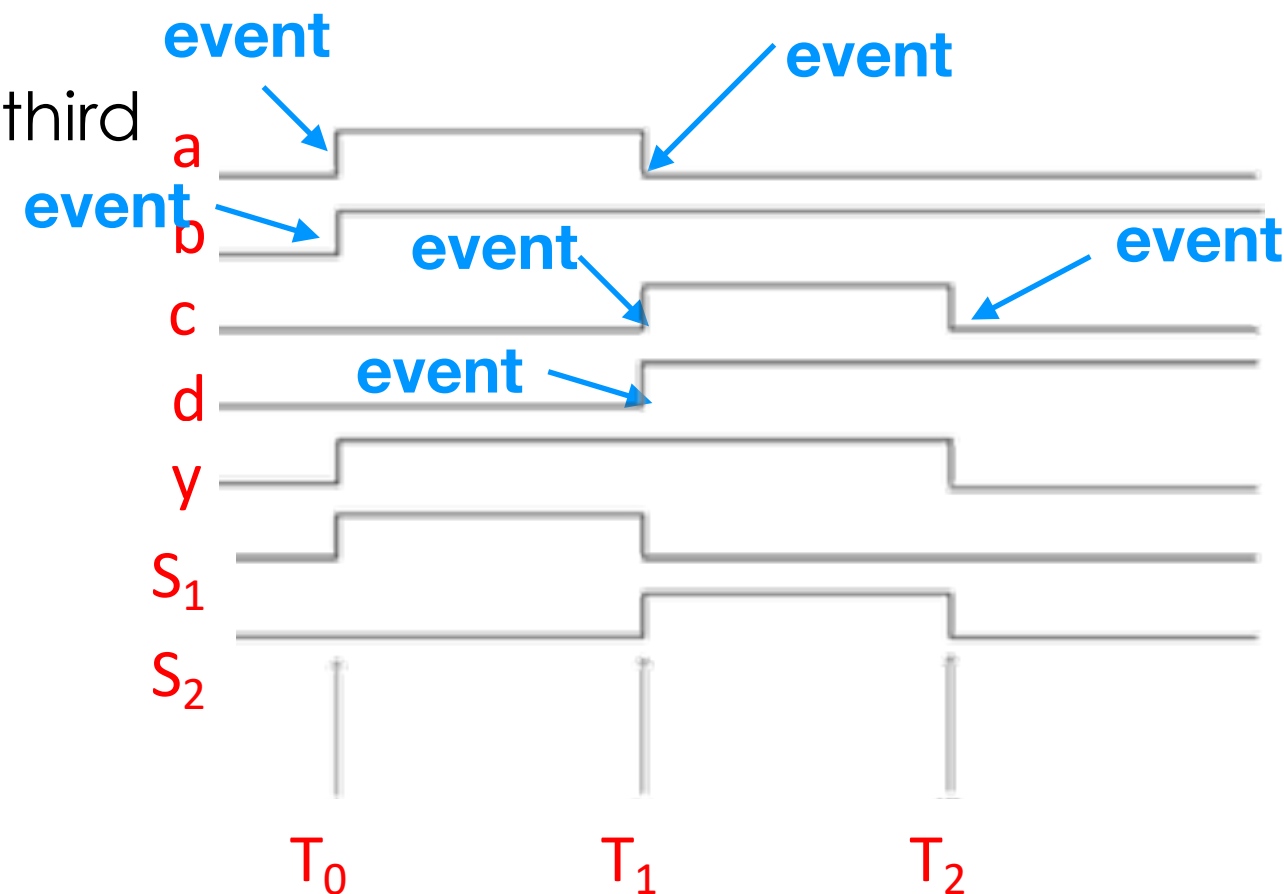
• Event at T_1

- On a, c and d
- Statement 1 and 2 executed first
- No delay; statement 3 executed
- S_1 and S_2 are updated

• Event at T_2

- On c; statement 2 executed

```
assign s1 = a & b;
assign s2 = c & d;
assign y  = s1 | s2;
```



CONSTANT DECLARATION

- Has same usage as other languages

- Declared by its type such as integer or time
- Use **=** to assign it a value
- the unit is simulation screen unit

```
time period =100; //verilog
parameter period = 100;
`define period 100;
```

- More data types:

- **integer** - 4-state Verilog data type, 32-bit signed integer
- **time** - 4-state Verilog data type, 64-bit unsigned integer

- Delay assignment

- Use #

```
assign #10 S1 = a & b;
```

AND-OR BLOCK WITH DELAY

```

module and_or(a,b,c,d,y);
    input a,b,c,d;
    output y;

    wire s1,s2;
    time dly = 10;
    assign #dly s1 = a & b;
    assign #dly s2 = c & d;
    assign #dly y  = s1 | s2;

endmodule

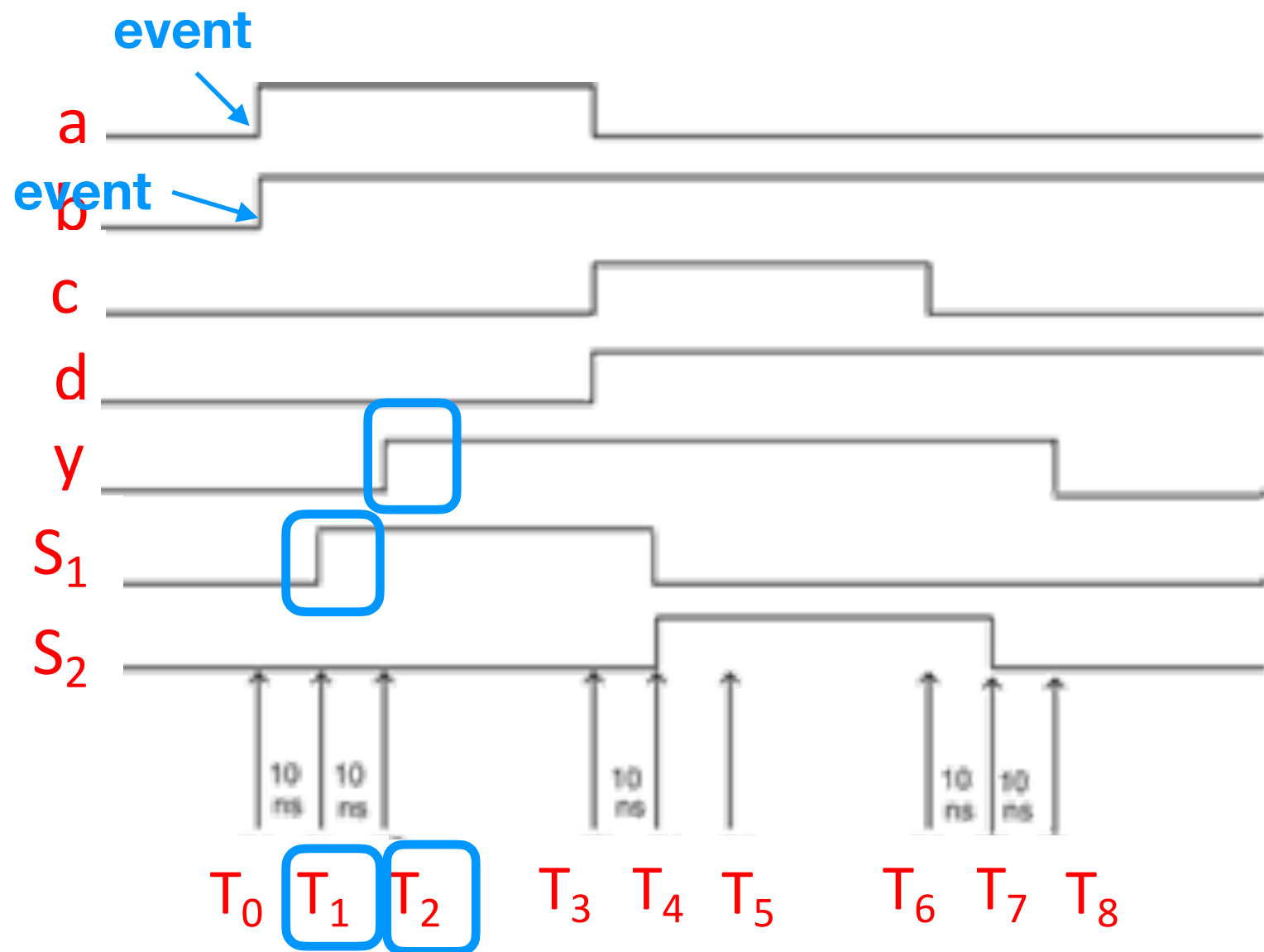
```

SIMULATION OF AND-OR BLOCK WITH DELAY

• Event at T_0

- On a and b; statement 1 is executed
- After dly (at T_1) S_1 is updated;
- Statement 3 is executed
- After dly (at T_2) y is updated

```
assign #dly s1 = a & b;
assign #dly s2 = c & d;
assign #dly y  = s1 | s2;
```



SIMULATION OF AND-OR BLOCK WITH DELAY

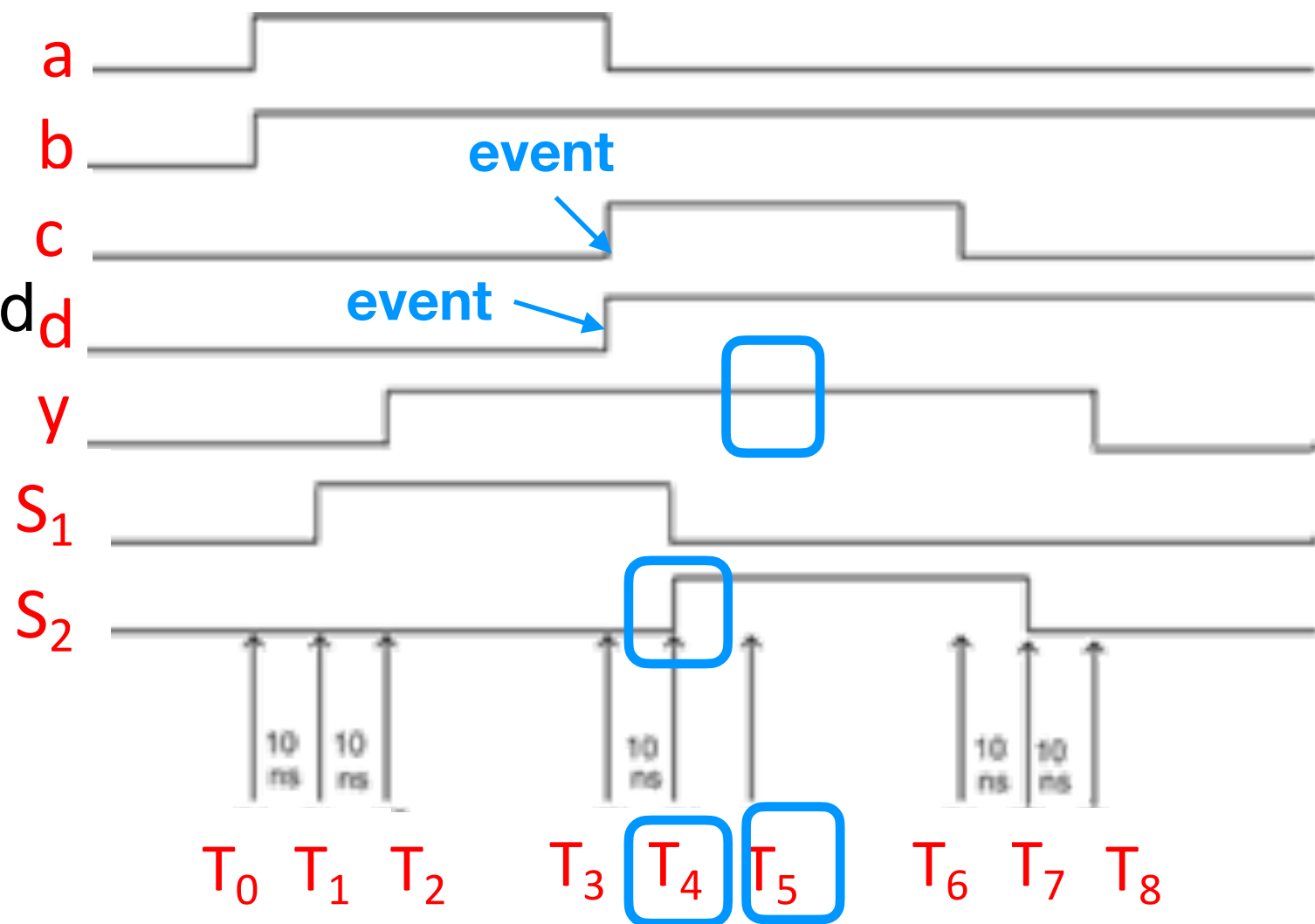
• Event at T_0

- On a and b; statement 1 is executed
- After dly (at T_1) S_1 is updated;
- Statement 3 is executed
- After dly (at T_2) y is updated

```
assign #dly s1 = a & b;
assign #dly s2 = c & d;
assign #dly y  = s1 | s2
```

• Event at T_3

- On a, c and d;
- Statement 1 & 2 are executed
- At T_4 , S_1 & S_2 are updated
- Statement 3 is executed
- At T_5 , y is updated



SIMULATION OF AND-OR BLOCK WITH DELAY

• Event at T_0

- On a and b; statement 1 is executed
- After dly (at T_1) S_1 is updated;
- Statement 3 is executed
- After dly (at T_2) y is updated

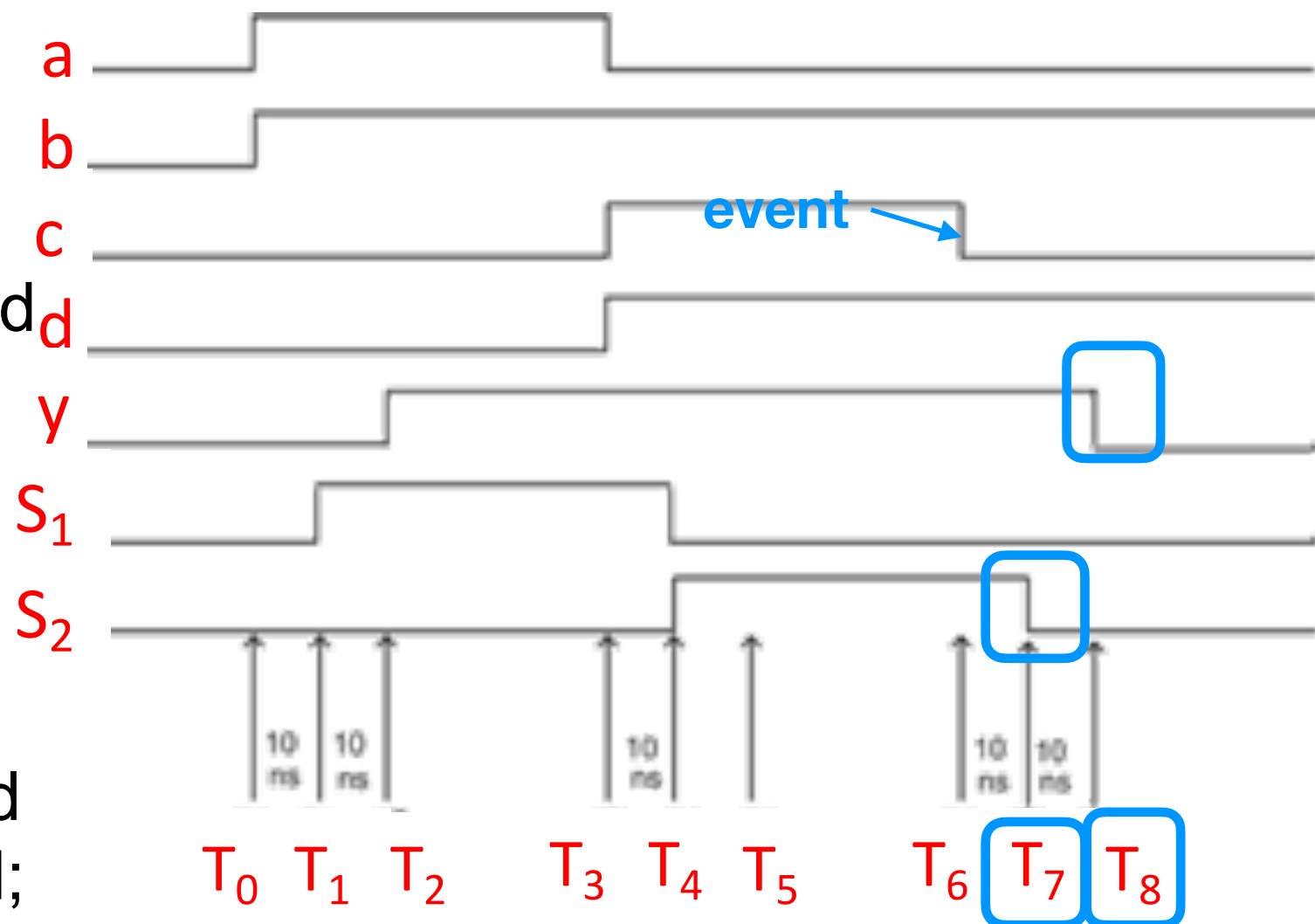
```
assign #dly s1 = a & b;
assign #dly s2 = c & d;
assign #dly y  = s1 | s2
```

• Event at T_3

- On a, c and d;
- Statement 1 & 2 are executed
- At T_4 , S_1 & S_2 are updated
- Statement 3 is executed
- At T_5 , y is updated

• Event at T_6

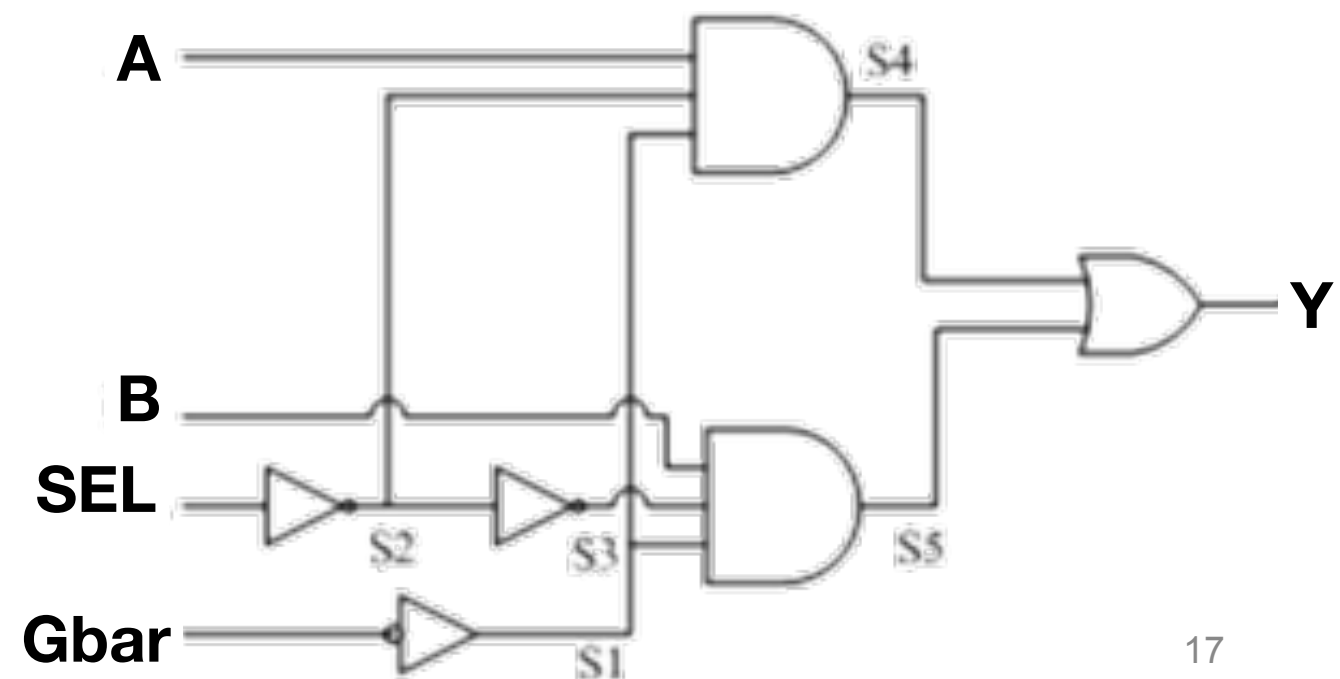
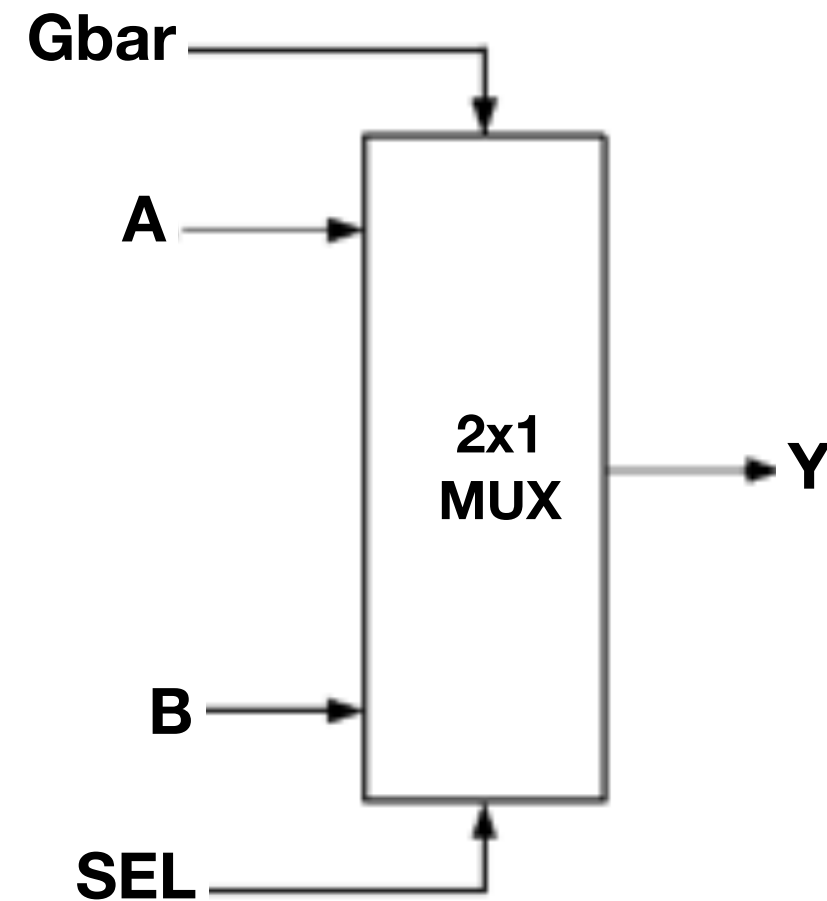
- on c; statement 2 is executed
- after dly (at T_7) S_1 is updated;
- statement 3 is executed
- after dly (at T_8) y is updated



DATA FLOW DESCRIPTION OF MUX 2X1

- 2x1 multiplexer
 - with active low enable
 - with constant delay
 - All gate delays are 7 ns

Input		Output
SEL	Gbar	Y
X	1	0
0	0	A
1	0	B

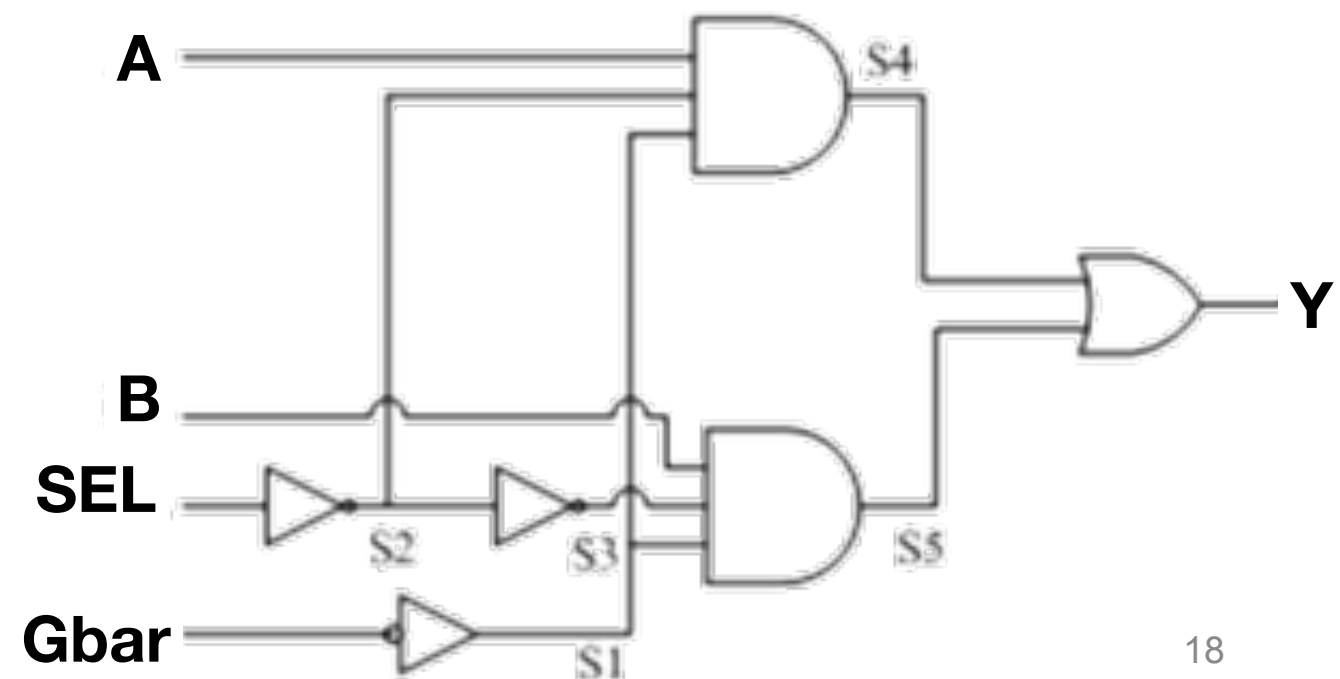
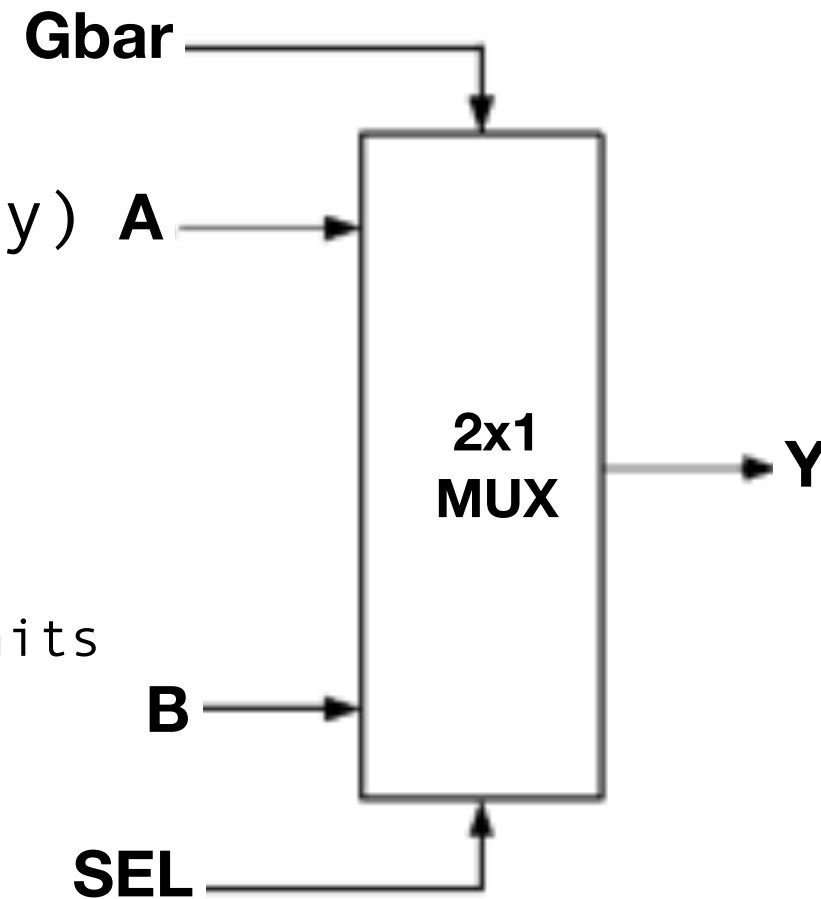


MUX 2X1- WITH DELAY CODE

```

module mux_enable(a,b,sel,gbar,y)
  input a,b,sel,gbar;
  output y;
  wire s1,s2,s3,s4,s5;
  time dly = 7; //simulation screen units
  assign #dly y=s4|s5;
  assign #dly s4=a & s2 & s1;
  assign #dly s5=b & s3 & s1;
  assign #dly s2= ~sel;
  assign #dly s3= ~s2;
  assign #dly s1= ~gbar;
endmodule

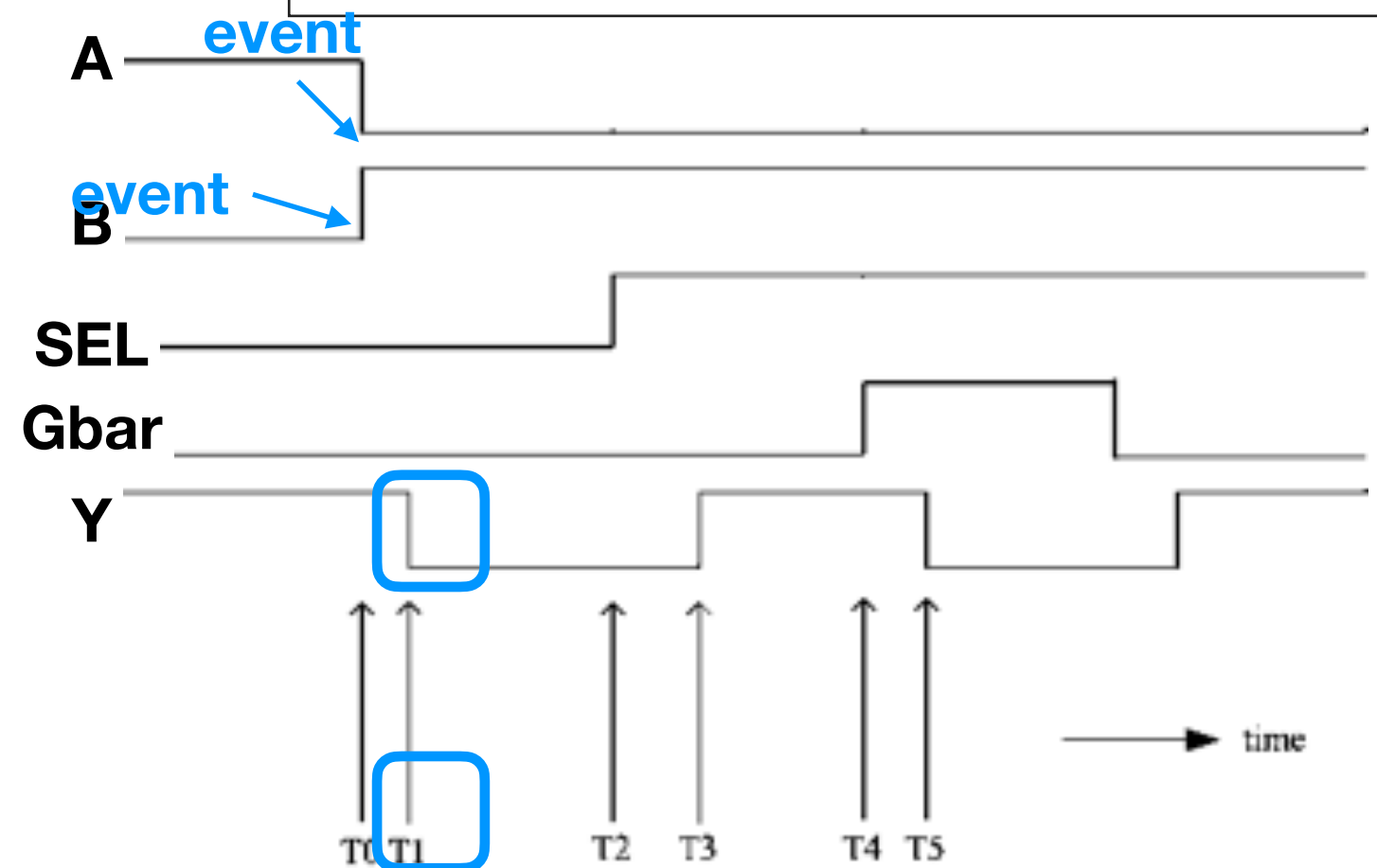
```



SIMULATION OF MUX 2X1

- If $T_0 = 100\text{ns}$
- $T_1 = 114\text{ns}$, why?

```
assign #dly y=s4|s5;
assign #dly s4=a & s2 & s1;
assign #dly s5=b & s3 & s1;
assign #dly s2= ~sel;
assign #dly s3= ~s2;
assign #dly s1= ~gbar;
```

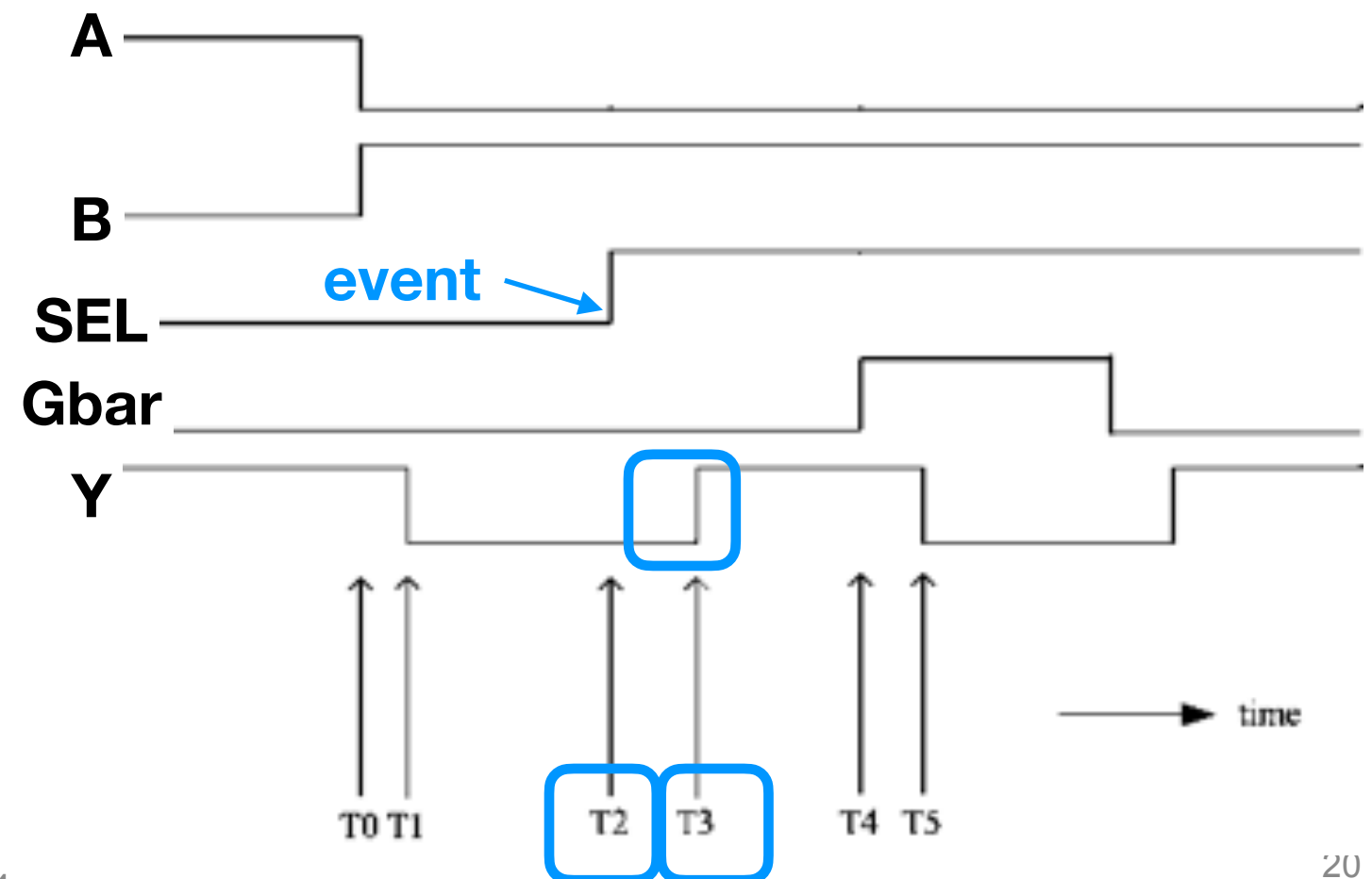


SIMULATION OF MUX 2X1

- If $T_0 = 100\text{ns}$
 - $T_1 = 114\text{ns}$, why?
- If $T_2 = 200\text{ns}$
 - $T_3 = 228\text{ns}$, why?

```

assign #dly y=s4|s5;
assign #dly s4=a & s2 & s1;
assign #dly s5=b & s3 & s1;
assign #dly s2= ~sel;
assign #dly s3= ~s2;
assign #dly s1= ~gbar;
    
```

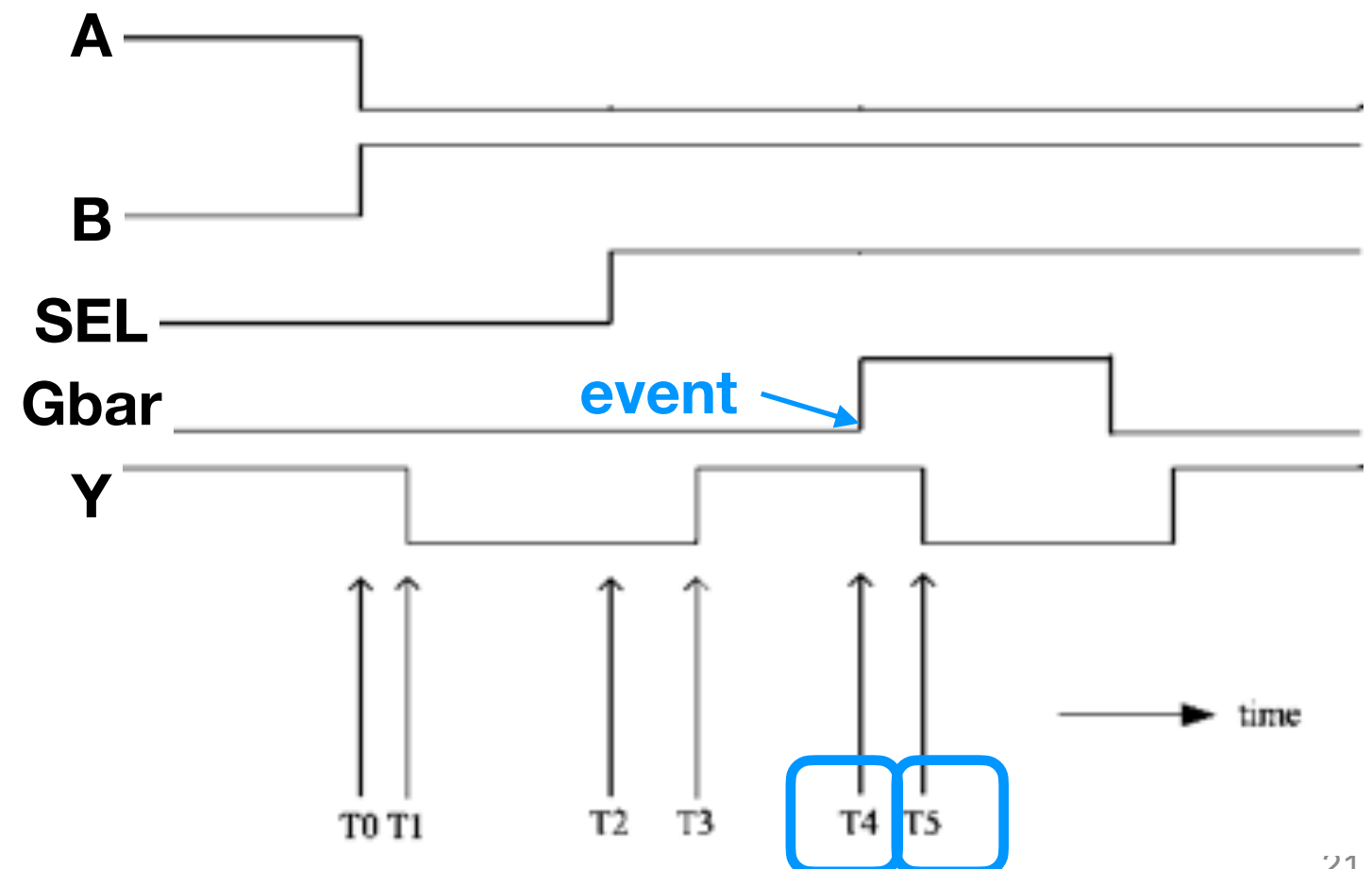


SIMULATION OF MUX 2X1

- If $T_0 = 100\text{ns}$
 - $T_1 = 114\text{ns}$, why?
- If $T_2 = 200\text{ns}$
 - $T_3 = 228\text{ns}$, why?
- If $T_4 = 300\text{ns}$
 - $T_5 = 321\text{ns}$, why?

```

assign #dly y=s4|s5;
assign #dly s4=a & s2 & s1;
assign #dly s5=b & s3 & s1;
assign #dly s2= ~sel;
assign #dly s3= ~s2;
assign #dly s1= ~gbar;
    
```



CONCURRENT IMPLEMENTATION

- Purely Concurrent Statement Syntax

- `assign`

- `assign out = in_a & in_b;`

- `?:`

- `mux = (addr==2'b00) ? i0 :
 ((addr == 2'b01) ? i1 :
 ((addr == 2'b10) ? i2 :
 ((addr == 2'b11) ? i3 :
 4'bz))) ;`

- `generate`

CONCURRENT IMPLEMENTATION: GENERATE

- **Syntax**

```
genvar i;
generate
for (i=0; i< MAX; i=i+1) begin
    [statements]
end
endgenerate
```


CONCURRENT IMPLEMENTATION: GENERATE

- Example on loop**

```
input [7:0] x;
```

```
input [15:0] y;
```

```
output [7:0] z;
```

```
genvar i;
```

```
generate
```

```
    for (i=0; i<=7; i=i+1) begin
```

```
        assign z[i] = x[i] & y[i+8];
```

```
    end
```

```
endgenerate
```

```
/*    Equivalent Example    */
assign z[0] = x[0] & y[0+8];
assign z[1] = x[1] & y[1+8];
assign z[2] = x[2] & y[2+8];
assign z[3] = x[3] & y[3+8];
assign z[4] = x[4] & y[4+8];
assign z[5] = x[5] & y[5+8];
assign z[6] = x[6] & y[6+8];
assign z[7] = x[7] & y[7+8];
```


LECTURE 4: OVERVIEW

- Behavioral Description
- Sequential statements
 - if, case, for, while, ...
 - Storage elements
 - DFF
 - JK FF
 - Binary counters
- Blocking vs. non-blocking assignment

Break 5 minutes



BEHAVIORAL DESCRIPTION

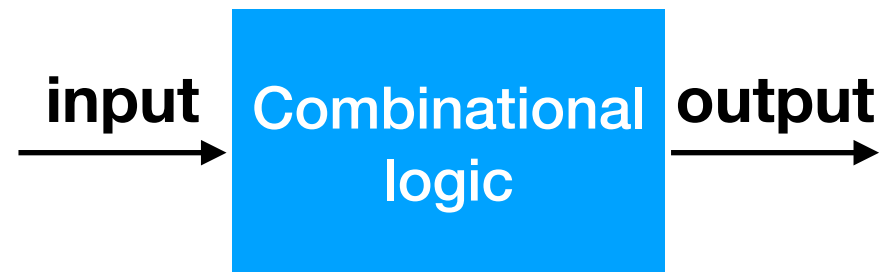
- How output **behaves** in response to input changes
- Details of logic diagram not needed
- Useful for
 - Combinational circuits modeling
 - Sequential circuits modeling
- Useful for complex designs
 - Arithmetic units
- Major behavioral description
 - **always/initial (Verilog)**

BEHAVIORAL DESCRIPTION

- How output **behaves** in response to input changes
- Details of logic diagram not needed
- Useful for
 - Combinational circuits modeling
 - Sequential circuits modeling
- Useful for complex designs
 - Arithmetic units
- Major behavioral description
 - **always/initial (Verilog)**

COMBINATIONAL VS. SEQUENTIAL CIRCUITS

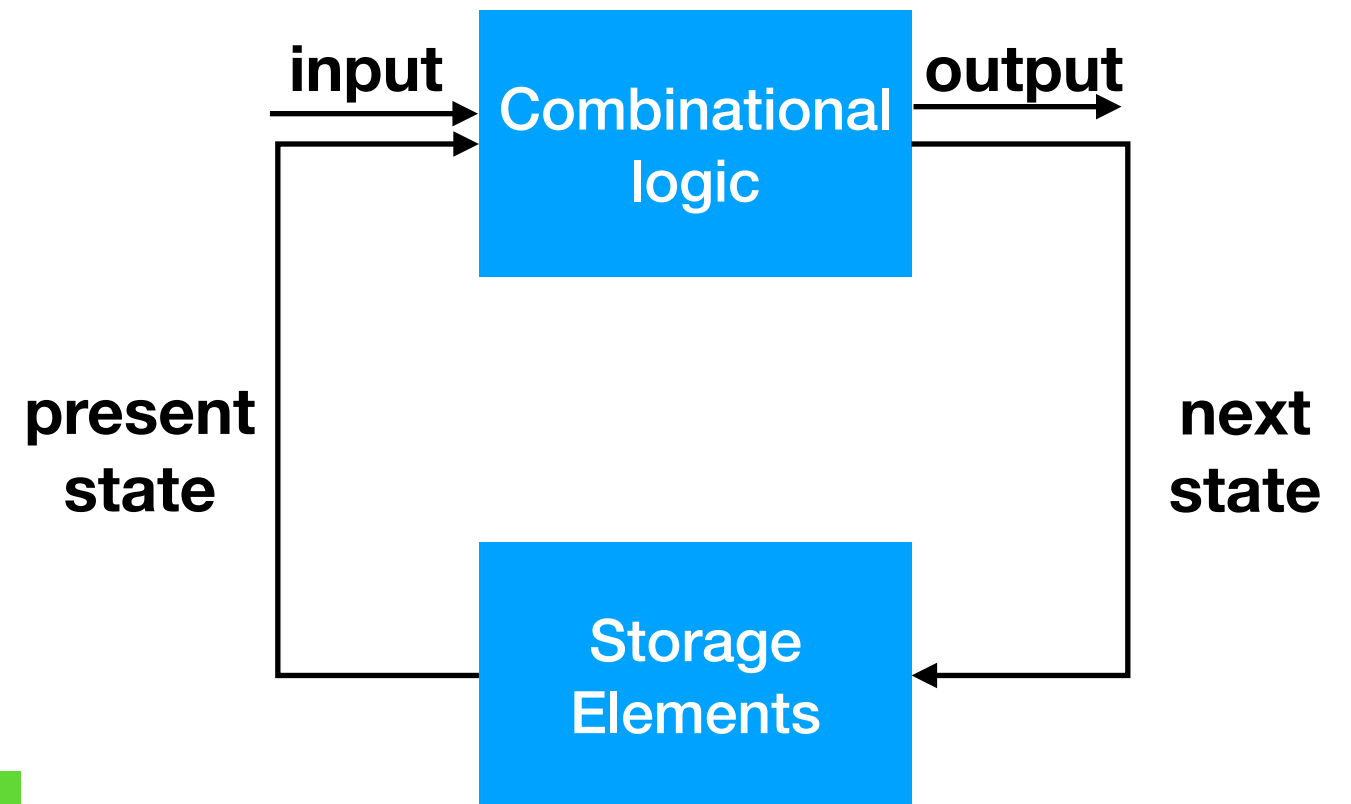
- Output depends on input



- **Decoder/Encoder**
- **Multiplexer/Demultiplexer**
- **Adder**
- **Address Decoder**

In principle, the system requires no memory and can be implemented using conventional logic gates

- Output depends on input and past state



- **Latches**
- **Flip-flops**
- **Registers**
- **Counters**
- **CPU controller**

BEHAVIORAL DESCRIPTION

- Basic statements of behavioral description
 - always/initial blocks
- Sequential statements
 - If
 - Case
 - Wait
 - Loop
- Blocking vs. non-blocking assignments

BEHAVIORAL DESCRIPTION

- Basic statements of behavioral description
 - **always/initial blocks**
- Sequential statements
 - If
 - Case
 - Wait
 - Loop
- Blocking vs. non-blocking assignments

ALWAYS

```
always @ (sensitivity_list)
begin
    //sequential_statements;
end
```

- Syntax: **always @ (*)**
- Sensitive to all signals inside always
- A variable should never be assigned a value in more than one always block, **why?**
- Any signal assigned inside always should be
 - reg
 - Integer
 - **wire cannot be assigned inside always**
- Execute **repeatedly** until the simulation is terminated

INITIAL

```
initial
begin
    //sequential_statements;
end
```

- Similar to always format
- No sensitivity list needed
- Executed **only once** at the **beginning** of simulation (at simulation time 0)
- Usually used
 - To initialize variables
 - Inside testbench
- Verilog guarantees that all initial blocks start before any always blocks

BEHAVIORAL DESCRIPTION

- Basic statements of behavioral description
 - always/initial blocks
- Sequential statements
 - If
 - Case
 - Wait
 - Loop
- Blocking vs. non-blocking assignments

IF STATEMENTS

- **If** statement is used to choose which statement should be executed depending on the **conditional** expression

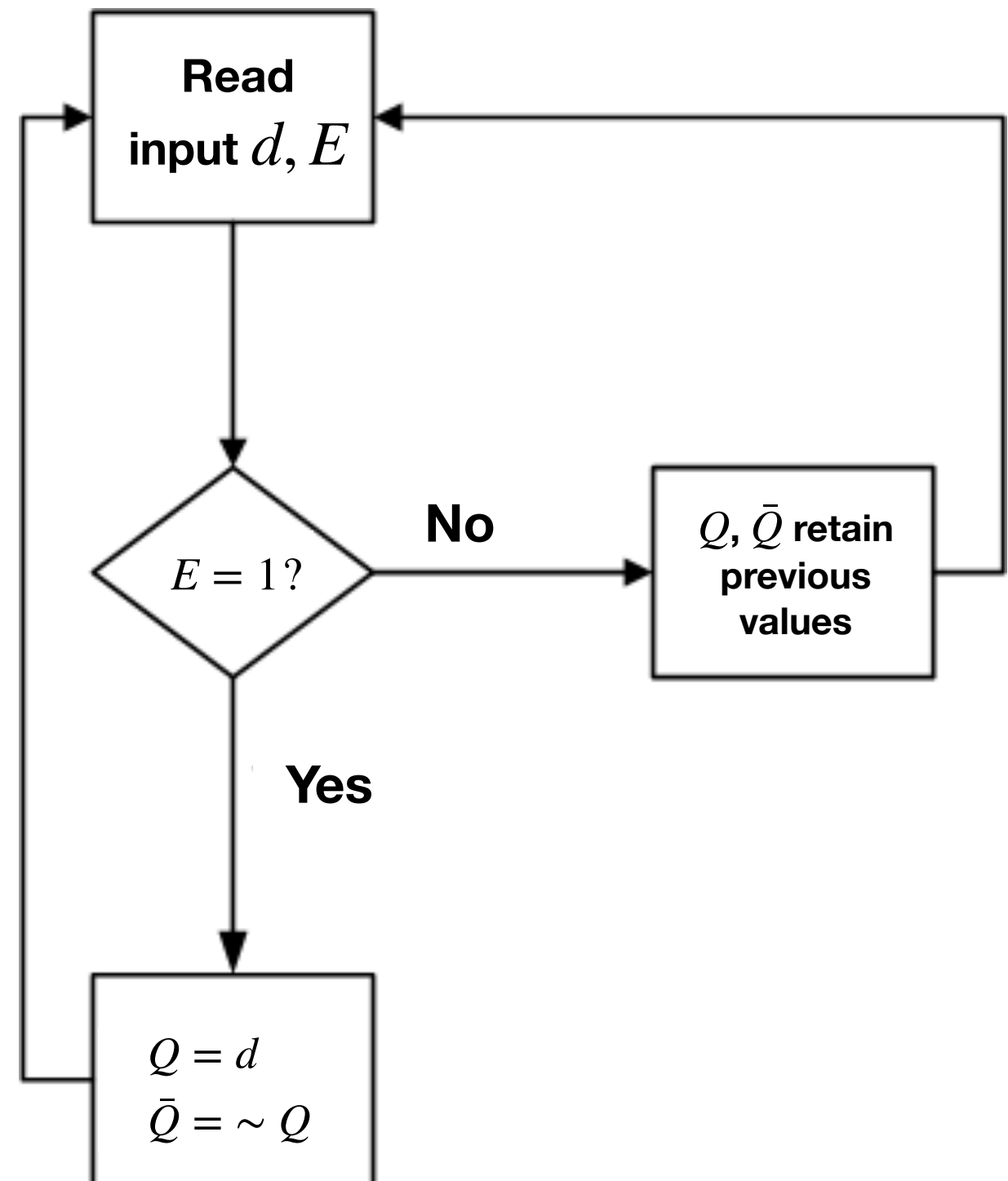
```
if (conditional expression) begin
    statement1;
    statement2;
end else if (conditional expression)
    statement3;
else
    statement4;
```

If only one statement
no need for begin/end

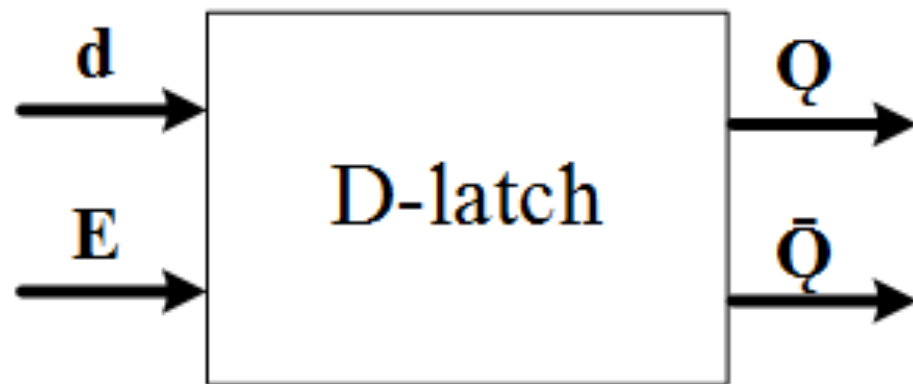
```
if (x==a && y==b)
    output = 1'b0;
else if (x==a && y==c)
    output = 1'b1;
else
    output = 1'bZ;
```

BEHAVIORAL DESCRIPTION EXAMPLE: D-LATCH

- D-latch
- Diagram
- Flowchart



BEHAVIORAL DESCRIPTION EXAMPLE: D-LATCH

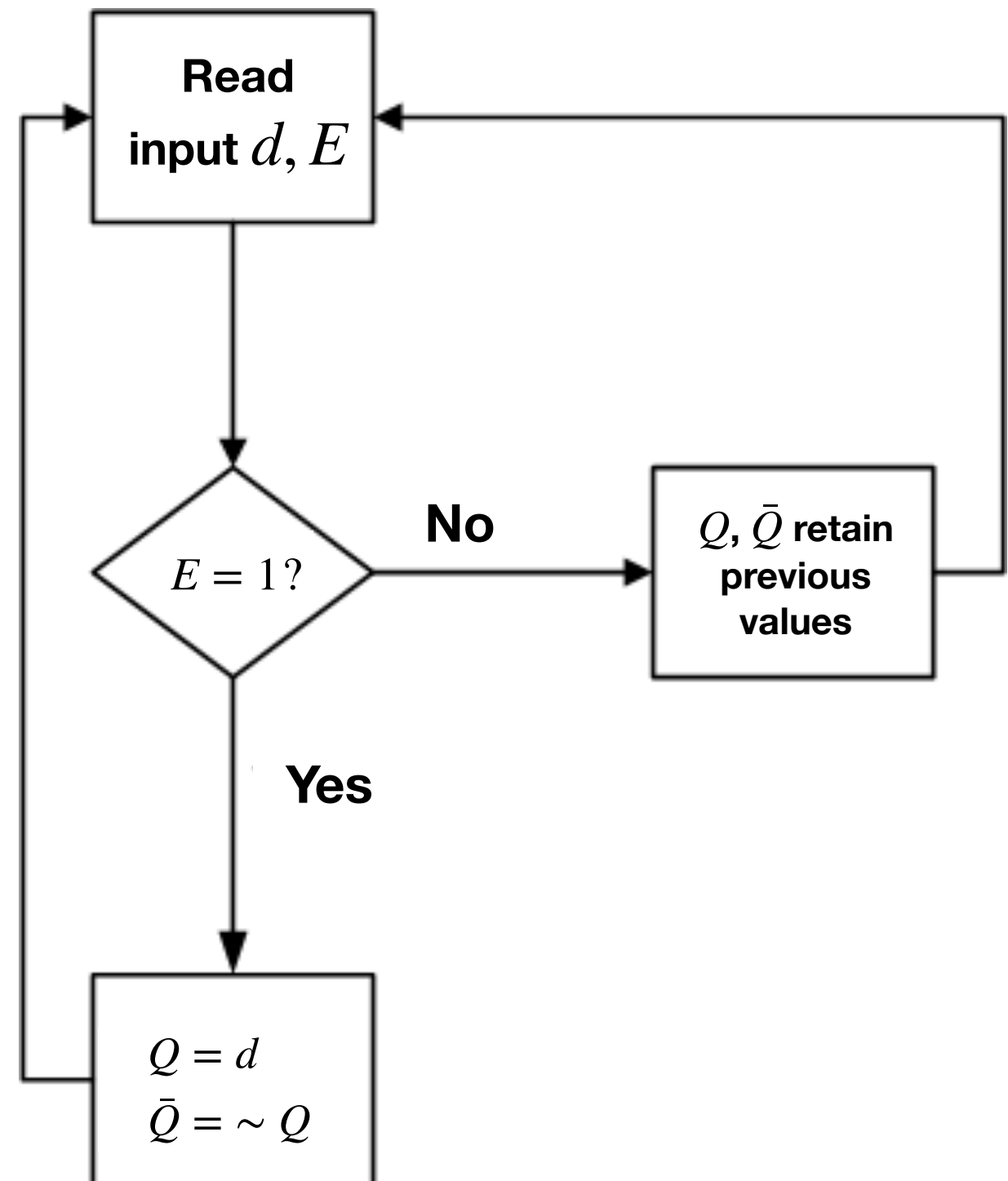


```

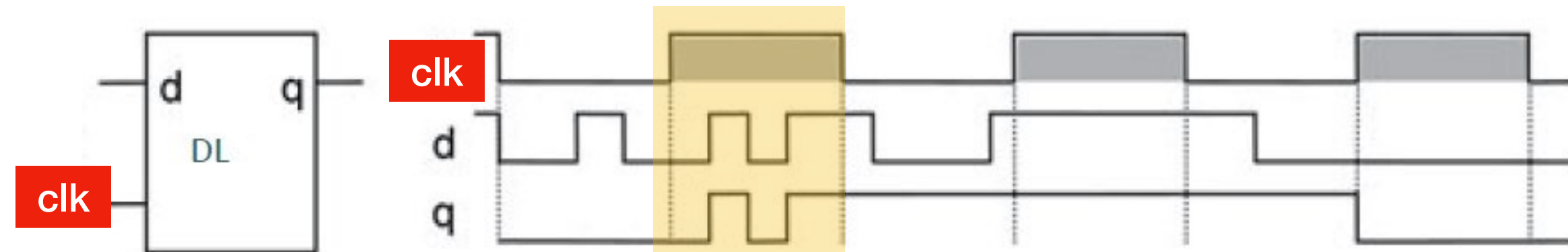
module D_latch (d,e,q,q_bar) ;
input d,e;
output reg q,q_bar;
  
```

```

always @(d,e)
begin
  if (e==1)
  begin
    q=d;
    q_bar = ~q;
  end
end
endmodule
  
```



BEHAVIORAL DESCRIPTION EXAMPLE: D-LATCH



```
module D_latch (d,e,q,q_bar) ;
input d,e;
output reg q,q_bar;
```

```
always @(d,e)
begin
    if (e==1)
    begin
        q=d;
        q_bar = ~q;
    end
end
endmodule
```

- Ports are defined by default as *wire*
- reg type before signal names are necessary if they are used in always block otherwise
COMPILATION ERROR

SEQUENTIAL STATEMENTS

- Sequential statements includes:
 - if
 - D-Latch example
 - case
 - wait
 - loop

CASE STATEMENTS

- **Case** statement:
 - The case statement selects for execution one of several alternative sequences of statements the alternative is chosen based on the value of associated expression.
 - Must cover all possible options
 - Simple but rigid!
 - It uses the verify operator === to do bitwise comparison
 - Everything must be explicitly coded

```
case (control-expression)
  value : begin statement1; end
  value : begin statement2; end
  default:begin default statement; end
endcase
```

```
case (sel)
  2'b00 : begin y = I1; z=I1&I2; end
  2'b01 : y = I2;
  2'b10 : y = I3;
  default: y = I4;
endcase
```

Single statement does not need begin/end

CASE, CASEX, CASEZ STATEMENTS

- Which statements will be executed if $a == 1'b0$?

```
case ( a )
1'b0 : statement1;
1'b1 : statement2;
1'bx : statement3;
1'bz : statement4;
endcase
```

```
casez ( a )
1'b0 : statement1;
1'b1 : statement2;
1'bx : statement3;
1'bz : statement4;
endcase
```

```
casex ( a )
1'b0 : statement1;
1'b1 : statement2;
1'bx : statement3;
1'bz : statement4;
endcase
```

- The **casez** statement treats high-impedance (z) values as don't-care values.
- The **casex** statement treats high-impedance (z) and unknown (x) values as don't care values.
- If any of the bits in the case expression or case item expression is a don't-care value then that bit position will be ignored.

CASE, CASEX, CASEZ STATEMENTS

- Which statements will be executed if $a == 1'b1$?

```
case ( a )
  1'b0 : statement1;
  1'b1 : statement2;
  1'bx : statement3;
  1'bz : statement4;
endcase
```

```
casez ( a )
  1'b0 : statement1;
  1'b1 : statement2;
  1'bx : statement3;
  1'bz : statement4;
endcase
```

```
casex ( a )
  1'b0 : statement1;
  1'b1 : statement2;
  1'bx : statement3;
  1'bz : statement4;
endcase
```

- The **casez** statement treats high-impedance (z) values as don't-care values.
- The **casex** statement treats high-impedance (z) and unknown (x) values as don't care values.
- If any of the bits in the case expression or case item expression is a don't-care value then that bit position will be ignored.

CASE, CASEX, CASEZ STATEMENTS

- Which statements will be executed if $a == 1'bz$?

```
case ( a )
  1'b0 : statement1;
  1'b1 : statement2;
  1'bx : statement3;
  1'bz : statement4;
endcase
```

```
casez ( a )
  1'b0 : statement1;
  1'b1 : statement2;
  1'bx : statement3;
  1'bz : statement4;
endcase
```

```
casex ( a )
  1'b0 : statement1;
  1'b1 : statement2;
  1'bx : statement3;
  1'bz : statement4;
endcase
```

- The **casez** statement treats high-impedance (z) values as don't-care values.
- The **casex** statement treats high-impedance (z) and unknown (x) values as don't care values.
- If any of the bits in the case expression or case item expression is a don't-care value then that bit position will be ignored.

CASE, CASEX, CASEZ STATEMENTS

- Which statements will be executed if $a == 1'bx$?

```
case ( a )
  1'b0 : statement1;
  1'b1 : statement2;
  1'bx : statement3;
  1'bz : statement4;
endcase
```

```
casez ( a )
  1'b0 : statement1;
  1'b1 : statement2;
  1'bx : statement3;
  1'bz : statement4;
endcase
```

```
casex ( a )
  1'b0 : statement1;
  1'b1 : statement2;
  1'bx : statement3;
  1'bz : statement4;
endcase
```

- The **casez** statement treats high-impedance (z) values as don't-care values.
- The **casex** statement treats high-impedance (z) and unknown (x) values as don't care values.
- If any of the bits in the case expression or case item expression is a don't-care value then that bit position will be ignored.